

Java DSA Course

Session: 2D Arrays (Matrices) in Java

Continuing after Arrays → Linear/Binary Search → Sorting

Duration	2 Hours (120 minutes)
Level	Absolute Beginners
Prerequisite	1D Arrays, Loops, Searching, Sorting (already covered)
Language	Java
Goal	Understand, declare, traverse, and manipulate 2D arrays (matrices)

Roadmap for Today's 2 Hours

- 0:00 – 0:10 → Recap + Why do we need 2D Arrays?
- 0:10 – 0:25 → Declaration, Memory Layout & Initialization
- 0:25 – 0:45 → Taking Input & Traversing a Matrix (nested loops, line-by-line)
- 0:45 – 1:10 → Core Operations: Sum, Row/Column Sum, Diagonal Sum, Transpose
- 1:10 – 1:30 → Searching an Element in a 2D Array
- 1:30 – 1:45 → Special Matrices: Identity Check, Symmetric Check
- 1:45 – 1:55 → Jagged Arrays (uneven 2D arrays)
- 1:55 – 2:00 → Practice Problems / Homework + Summary

1. Quick Recap

Before jumping in, remember where we left off: a 1D array like `int[] arr = new int[5];` stores elements in a single row, one after another in memory, and we access them using a single index — `arr[0]`, `arr[1]`, and so on. We used this to learn Linear Search, Binary Search, and Sorting techniques (Bubble, Selection, etc.).

Why do we need a 2D Array?

Think about real data that isn't just a single line of values — a classroom seating chart, a chess board, an image made of pixels, or a table of student marks (rows = students, columns = subjects). A single-index (1D) array cannot represent this naturally. We need something with **rows and columns** — that's exactly what a **2D array (matrix)** gives us.

■ Think of a 2D array as an Excel sheet: you need a row number AND a column number to reach one cell.

2. Declaring a 2D Array in Java

```
// Syntax
dataType[][] arrayName = new dataType[rows][columns];

// Example: a matrix with 3 rows and 4 columns
int[][] matrix = new int[3][4];
```

- **dataType[][]** — the double square brackets tell Java this is a 2D array (an array of arrays).
- **new dataType[rows][columns]** — this actually allocates memory: here it creates 3 rows, and each row has 4 columns (so $3 \times 4 = 12$ int slots).
- By default, all values in an `int[][]` are initialized to **0** (for boolean it's false, for objects it's null).

How Java Stores a 2D Array in Memory

In Java, a 2D array is technically an **array of arrays**. The outer array holds references, and each reference points to a separate 1D array (a row). This is different from languages like C, where a 2D array is one continuous memory block. This is exactly why Java allows **jagged arrays** — rows of different lengths — which we'll see later.

Initializing with Values Directly

```
int[][] marks = {
    {90, 85, 78}, // row 0 → student 1's marks in 3 subjects
    {60, 70, 65}, // row 1 → student 2's marks
    {88, 92, 81}  // row 2 → student 3's marks
};
```

- Each `{ }` inside the outer braces is one row.
- **marks[0]** refers to the entire first row: `{90, 85, 78}`.
- **marks[0][1]** refers to a single cell — row 0, column 1 → value 85.
- Rows are indexed 0 to (rows-1), columns are indexed 0 to (columns-1), same zero-based rule as 1D arrays.

■ `marks.length` gives the number of rows. `marks[0].length` gives the number of columns in row 0.

3. Taking Matrix Input from the User

```
import java.util.Scanner;

public class MatrixInput {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of rows: ");
        int rows = sc.nextInt();
        System.out.print("Enter number of columns: ");
        int cols = sc.nextInt();

        int[][] matrix = new int[rows][cols];

        // Nested loop to READ values
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                System.out.print("Enter element [" + i + "][" + j + "]: ");
                matrix[i][j] = sc.nextInt();
            }
        }

        System.out.println("Matrix successfully filled!");
    }
}
```

Line-by-Line Explanation

- **Scanner sc = new Scanner(System.in);** — creates an object that lets us read input typed by the user from the keyboard.
- **int rows = sc.nextInt();** — reads one integer typed by the user and stores it as the number of rows. Same for **cols**.
- **int[][] matrix = new int[rows][cols];** — now the array size is decided at run-time based on what the user typed (this is called dynamic sizing).
- **for (int i = 0; i < rows; i++)** — the **outer loop** moves row by row. Variable **i** represents the current row number.
- **for (int j = 0; j < cols; j++)** — the **inner loop** moves column by column *within* the current row. Variable **j** represents the current column number.
- For every combination of (i, j), we ask the user for a value and store it at **matrix[i][j]** — this is how every single cell gets filled, row after row.
- **Golden Rule:** outer loop = rows, inner loop = columns. This pattern repeats in almost every 2D array program.

4. Traversing (Printing) a Matrix

```
// Nested loop to PRINT the matrix in grid format
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        System.out.print(matrix[i][j] + "\t");    // \t = tab space for alignment
    }
    System.out.println();    // move to next line after finishing one row
}
```

}

- The inner loop prints all columns of the current row, side by side, using **System.out.print** (no new line) plus a tab character for neat spacing.
- Once the inner loop finishes (one full row is printed), we call **System.out.println()** with nothing inside — this simply moves the cursor to the next line, so the next row starts fresh.
- This 'print all columns, then move to next line' pattern is exactly how we visualize a matrix as a grid on screen.

■ Whenever you're stuck with 2D array logic, mentally trace it as: outer loop = 'which row am I on', inner loop = 'walk across that row'.

5. Sum of All Elements

```
int sum = 0;
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        sum = sum + matrix[i][j];
    }
}
System.out.println("Sum of all elements: " + sum);
```

- We start with **sum = 0** as an accumulator (a variable that keeps adding up values).
- As the nested loop visits every cell exactly once, we add that cell's value into **sum**.
- After both loops finish, every one of the rows × cols elements has been added exactly once.

6. Row-wise and Column-wise Sum

```
// Row-wise sum
for (int i = 0; i < rows; i++) {
    int rowSum = 0;
    for (int j = 0; j < cols; j++) {
        rowSum += matrix[i][j];
    }
    System.out.println("Sum of row " + i + " = " + rowSum);
}

// Column-wise sum
for (int j = 0; j < cols; j++) {
    int colSum = 0;
    for (int i = 0; i < rows; i++) {
        colSum += matrix[i][j];
    }
    System.out.println("Sum of column " + j + " = " + colSum);
}
```

- For **row sum**: notice **rowSum** is declared *inside* the outer loop — so it resets to 0 for every new row, then the inner loop adds up that one row only.
- For **column sum**: we flip the loop order! The outer loop now walks column-by-column (**j**), and the inner loop walks down the rows (**i**) for that fixed column — this is a common beginner confusion point, so trace it slowly on paper once.

7. Diagonal Sum (Only for Square Matrices)

```
// Works only when rows == cols
int diagonalSum = 0;
for (int i = 0; i < rows; i++) {
    diagonalSum += matrix[i][i];
}
System.out.println("Diagonal sum: " + diagonalSum);
```

- Notice we use a **single loop**, not nested — because the main diagonal always has row index == column index, i.e., cells (0,0), (1,1), (2,2)...
- So **matrix[i][i]** alone walks the entire diagonal without needing an inner loop.

8. Transpose of a Matrix

The transpose of a matrix flips it over its diagonal — rows become columns and columns become rows. If the original matrix is $r \times c$, the transpose will be $c \times r$.

```
int[][] transpose = new int[cols][rows];    // dimensions swapped!
```

```
for (int i = 0; i < rows; i++) {  
    for (int j = 0; j < cols; j++) {  
        transpose[j][i] = matrix[i][j];  
    }  
}
```

- We create a new matrix called **transpose** whose size is **cols × rows** (swapped) — since transpose flips dimensions.
- The key line is **transpose[j][i] = matrix[i][j]** — the row and column indices are simply swapped when copying each value across.

9. Linear Search in a Matrix

Just like we did linear search on a 1D array (checking each element one by one), we can do the same on a 2D array — we just need two loops instead of one to visit every cell.

```
int target = 65;
boolean found = false;

for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        if (matrix[i][j] == target) {
            System.out.println("Found " + target + " at position (" + i + ", " + j + ")");
            found = true;
        }
    }
}

if (!found) {
    System.out.println(target + " not found in the matrix.");
}
```

- **found** is a flag (boolean) we use to remember whether we located the value anywhere in the matrix.
- Both loops together guarantee we check every single cell exactly once, same idea as full traversal, but now with an **if** check inside.
- When a match is found, we print its exact (row, column) position — this is the real advantage of a 2D array: position carries meaning (unlike 1D, where index alone told you 'where').

■ This is $O(\text{rows} \times \text{cols})$ time complexity — same growth idea as linear search, just across a grid instead of a line.

10. Checking if a Matrix is an Identity Matrix

An Identity Matrix is a square matrix where all diagonal elements are 1, and every other element is 0.

```
boolean isIdentity = true;

for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        if (i == j && matrix[i][j] != 1) {
            isIdentity = false;
        }
        if (i != j && matrix[i][j] != 0) {
            isIdentity = false;
        }
    }
}
System.out.println("Is Identity Matrix? " + isIdentity);
```

- `i == j` means we are on a diagonal cell — it must be 1, otherwise the condition fails and we set the flag to false.
- `i != j` means we are off the diagonal — it must be 0, otherwise, again, it's not an identity matrix.
- We never 'break' early here to keep the logic simple for beginners — it still checks every cell, which is fine for small matrices.

11. Checking if a Matrix is Symmetric

A matrix is symmetric if it is equal to its own transpose, meaning `matrix[i][j] == matrix[j][i]` for every cell.

```
boolean isSymmetric = true;

for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        if (matrix[i][j] != matrix[j][i]) {
            isSymmetric = false;
        }
    }
}
System.out.println("Is Symmetric? " + isSymmetric);
```

- We simply compare each cell with its 'mirror' cell across the diagonal: `matrix[i][j]` vs `matrix[j][i]`.
- This check only makes sense for a **square** matrix (`rows == cols`) — otherwise `matrix[j][i]` may not even exist.

12. Jagged Arrays (Arrays of Arrays with Different Lengths)

Because Java stores a 2D array as an array of row-references (not one solid block), rows are allowed to have different lengths. This is called a **jagged array**.

```
int[][] jagged = new int[3][];    // 3 rows, columns not fixed yet

jagged[0] = new int[]{1, 2};      // row 0 has 2 columns
jagged[1] = new int[]{3, 4, 5, 6}; // row 1 has 4 columns
jagged[2] = new int[]{7};         // row 2 has 1 column

for (int i = 0; i < jagged.length; i++) {
    for (int j = 0; j < jagged[i].length; j++) {
        System.out.print(jagged[i][j] + " ");
    }
    System.out.println();
}
```

- **new int[3][]** — notice the second bracket is empty; we're only fixing the number of rows (3), each row's length is decided later.
- Each row is then created separately, as its own independent 1D array, and can have any length we want.
- In the print loop, we use **jagged[i].length** (not a fixed **cols** variable) inside the inner loop, because each row's length is different now.

■ This is why, in true Java 2D arrays, the correct way to get columns is always `row.length` — `arr[i].length` — not one fixed number, in case the array is jagged.

13. Practice Problems (Try Before Next Class)

- 1. Write a program to add two matrices of the same size and print the result matrix.
- 2. Write a program to multiply two matrices ($A \times B$), where columns of A == rows of B.
- 3. Print only the boundary (border) elements of a matrix.
- 4. Find the largest and smallest element in a matrix, along with their positions.
- 5. Rotate a square matrix by 90 degrees clockwise.
- 6. Given a sorted matrix (each row and column sorted), search for a target efficiently (hint: start from top-right corner).

14. Summary of What We Learned Today

- A 2D array represents data with rows and columns — declared as **type[][] name = new type[rows][cols]**.
- Java implements 2D arrays as an **array of arrays** (row references), which is why jagged arrays are possible.
- The pattern **outer loop = rows, inner loop = columns** is used for input, printing, sum, and searching.
- Diagonal-based operations (diagonal sum, identity check, symmetric check) use the relationship between **i** and **j**.
- Transpose swaps dimensions and swaps indices: **transpose[j][i] = matrix[i][j]**.
- Next session: Matrix Addition & Multiplication, and 2D array-based DSA problems (Spiral traversal, Rotate matrix).

End of Session — Great work! See you in the next class for Matrix Addition & Multiplication.